Attack Detection, Analysis and Mitigation

Project: Network Packet Analyzer and Intrusion Detection System (Python, Scapy, Flask, MongoDB, React)

Author: Dondapati Polireddy

Attack	Rule	Severity
Port Scan	15 unique ports from one IP in 5 s	HIGH
SYN Flood	100 SYN packets from one IP in 10 s	HIGH
ARP Spoof	1 MAC claiming 3 or more IPs	MEDIUM

Port Scan

1. Real example

```
Attacker (192.168.31.77) runs:
  nmap -sS 192.168.31.156
Sends SYN packets to ports 1,2,3 ... 1000 within seconds.
Simulated in project: python simulate_attacks.py (ports 1-24).
```

2. How the IDS detects it

```
check_port_scan(src_ip, dest_port, time):
  - skip if src_ip in WHITELIST
  - reset set if more than 5 s since window start
  - add dest_port to set for that IP
  - if len(set) >= 15 -> save_alert(PORT_SCAN, HIGH), clear set
```

3. Alert stored in MongoDB

```
{ type: 'PORT_SCAN', src_ip: '192.168.31.77', severity: 'HIGH',
  description: 'PORT SCAN DETECTED from 192.168.31.77 hit 15 ports' }
```

4. How I analyze it

- Open Alerts page; note source IP, time and number of ports hit.
- Check Packets page filtered by that IP: many different destination ports, few packets each = scan pattern.
- Decide if the IP is internal (misconfigured tool, admin) or external (reconnaissance).
- Ask the AI assistant: 'Which IP is most active?' to confirm volume.

5. Mitigation steps

- Block the source IP at the firewall (Windows: netsh advfirewall firewall add rule name=Block dir=in action=block remoteip=IP).
- Close unused ports and disable unneeded services.
- Allow only required ports; rate-limit new connections per IP.
- Add a legitimate scanner (e.g. admin vulnerability scanner) to WHITELIST.
- Keep logs and CSV export as evidence.

SYN Flood

1. Real example

```
Attacker sends a very large number of SYN packets to port 80 and never completes the handshake (no ACK),
Simulated: 110 SYN packets to port 80 using Scapy.
```

2. How the IDS detects it

```
syn_flood_scan(src_ip, flags, time):
  - ignore if flags != 'S' (only pure SYN counted)
  - reset counter if more than 10 s since window start
  - counter += 1
  - if counter >= 100 -> save_alert(SYN_FLOOD, HIGH), reset counter
```

3. Alert stored in MongoDB

```
{ type: 'SYN_FLOOD', src_ip: '10.0.0.4', severity: 'HIGH',
  description: 'SYN FLOOD DETECTED from 10.0.0.4 sent 100 SYN packets' }
```

4. How I analyze it

- Check how many SYNs vs normal ACK packets come from the IP: flood = many SYN, almost no ACK.
- Look at the dashboard line chart: sudden spike in packets over time.
- Check whether the destination service is slow or unreachable.
- Check if source IPs are spoofed or many (DDoS) or a single host (DoS).

5. Mitigation steps

- Enable SYN cookies on the server (Linux: sysctl -w net.ipv4.tcp_syncookies=1).
- Rate-limit SYN packets per IP in the firewall.
- Block the offending IP; for DDoS use upstream filtering / CDN.
- Reduce SYN timeout and increase backlog queue size.
- Document the incident and tune the threshold if needed.

ARP Spoofing

1. Real example

```
Attacker's MAC aa:bb:cc:dd:ee:ff sends ARP replies claiming to be 192.168.1.1 (router), then 192.168.1.2,
Simulated using Scapy Ether/ARP with sendp.
```

2. How the IDS detects it

```
check_arp_spoof(mac, src_ip, time):
  - skip whitelisted IPs
  - arp_spoof_scanning[mac].add(src_ip)
  - if len(set) >= 3 -> save_alert(ARP_SPOOF, MEDIUM), clear set
Normal device: one MAC maps to one IP, so no alert.
```

3. Alert stored in MongoDB

```
{ type: 'ARP_SPOOF', src_ip: '192.168.1.3', severity: 'MEDIUM',
  description: 'ARP SPOOF: MAC aa:bb:cc:dd:ee:ff claims 3 IPs' }
```

4. How I analyze it

- Run 'arp -a' on a victim: two or more IPs show the same MAC address.
- Verify the real router MAC against the router label or admin page.
- Identify the physical device owning the attacker MAC (switch port table).
- Check packets for traffic being redirected through the attacker.

5. Mitigation steps

- Disconnect or isolate the attacker's port/device.
- Add static ARP entries for the gateway (arp -s IP MAC).
- Enable Dynamic ARP Inspection and DHCP snooping on managed switches.
- Use VLAN segmentation and encrypted protocols (HTTPS, SSH, VPN).
- Flush ARP cache on victims (arp -d *) after removal.

How I Handle False Positives

A false positive is a normal activity flagged as an attack. Too many make the alerts useless, so I handled them in steps:

1. **Found the problem.** Thousands of false 'hit 1 ports' alerts appeared. Cause: a bug where the alert fired on a wrong length check.
2. **Fixed the logic.** Used len(port_scan_tracking[src_ip]) against the threshold of 15 and replaced string time trackers with dictionaries.
3. **Whitelist.** Trusted IPs (router 192.168.31.1, my laptop, localhost) are skipped in all three detectors.
4. **Time window.** Counters reset after 5 s (port scan) or 10 s (SYN), so slow normal traffic never accumulates.
5. **Thresholds.** 15 ports and 100 SYNs are well above normal browsing; they can be tuned per network.
6. **Only count SYN.** Normal ACK/data packets are ignored by the flood rule (test 6).
7. **Reset after alert.** Tracker clears after an alert, avoiding repeated duplicate alerts (test 9).
8. **Unit tests.** 10 tests prove normal traffic gives no alert and attacks do.

Short interview answer

"My IDS uses sliding time windows to detect port scans, SYN floods and ARP spoofing. I analyze each alert using the packets page and source IP, then mitigate by blocking the IP, enabling SYN cookies or static ARP. I reduce false positives with a whitelist, tuned thresholds, time-window resets and 10 unit tests; I also fixed a real bug that caused thousands of false alerts."